Lost in Translation - Group Project Brief

- JavaScript, PHP, and Web APIs
- Team size: 3 students
- Duration: 1 week

1. The task

You are tasked to implement a web game built around deliberate mistranslation.

The application takes a short English sentence, pushes it through a chain of six different languages using a translation API, then brings it back to English. What comes out the other side is usually mangled beyond recognition. That mangled output is the puzzle.

Players are shown only the wreckage. They race to guess the original sentence. When the round ends, the application replays the full chain step by step so everyone can watch exactly where the meaning fell apart. The most thoroughly destroyed sentences are preserved in a permanent Hall of Fame.

Your system must handle the translation chain on the server, cache aggressively so you do not exhaust a free API quota, score guesses with fuzzy matching rather than exact string comparison, and present the whole thing in a browser interface that makes the degradation fun to watch.

2. Team roles

Student A: Translation service. Owns the API client, the language chain runner, the cache layer (cache each hop against its text and language pair, so a sentence that has already been through the chain costs no further API calls), retry and error handling, and the quota accounting. Delivers a single PHP function that the rest of the team calls and otherwise never touches the external API.

Student B: Game logic and data. Owns the database schema, the round lifecycle (a round is generated, opens for guessing on a fixed timer, then closes and is revealed), guess submission, the similarity scoring algorithm, the mangle score, and the Hall of Fame queries. The two scores are defined as follows.

- Similarity: lowercase both strings, strip punctuation, collapse whitespace, then compare using Levenshtein distance or word overlap, normalized to a 0 to 1 scale. A guess of 0.85 and above is correct, 0.60 to 0.85 is close, anything lower is wrong. A correct guess

is worth 100 points, less one point per second elapsed in the round and less 30 points per hint used.
- Mangle score: the same similarity measure applied to the original seed against the final returned English, reported from 0 to 100, where higher means more destroyed. This is what ranks the Hall of Fame.

Student C: Frontend. Owns the game interface, the reveal animation, live round state (the page polls a PHP endpoint every couple of seconds for the timer and the current state, so the server owns the clock, not the browser), the Hall of Fame view, and all client-side validation.

3. The translation API

You may choose either provider. Option A is simpler and free. Option B produces different and arguably more interesting behaviour, and requires a key. You may also implement both behind a common interface, which is the strongest answer.

Whichever you choose, all API calls happen in PHP. No exceptions. A translation call originating from browser JavaScript is an automatic deduction.

Option A: MyMemory

Free, no key required, no signup.

Endpoint: https://api.mymemory.translated.net/get
Method: GET
Example: https://api.mymemory.translated.net/get?q=Hello%20world&langpair=en|ja

Documentation: https://mymemory.translated.net/doc/spec.php Usage limits: https://mymemory.translated.net/doc/usagelimits.php

Free anonymous use is capped at roughly 5,000 characters per day, counted per IP address. Passing a valid email address as a `de` parameter raises that to roughly 50,000. One chain is seven calls, so an uncached team will run dry within the hour, and a team sharing a network shares a single quota.

Option B: Claude (Anthropic Messages API)

Requires an API key. Create your own at https://platform.claude.com.

Endpoint: https://api.anthropic.com/v1/messages
Method: POST

Use `claude-haiku-4-5-20251001`. It is the cheapest and fastest model in the family, and translation of a single short sentence is exactly the kind of task it is built for. At current pricing (roughly one dollar per million input tokens) a full semester of development will cost a few cents.

Documentation: https://platform.claude.com/docs/en/api/overview

4. The language chain

The default chain must pass through six intermediate languages before returning to English. That is seven API calls per sentence.

en -> ja -> ar -> fi -> sw -> hu -> ko -> en

These languages are chosen deliberately. Japanese, Korean, Finnish, Hungarian, Arabic, and Swahili are either non-Indo-European or morphologically rich, and they handle articles, tense, plurals, and word order very differently from English. A chain through French, Spanish, and Italian will return your sentence almost unharmed, which is a nice thing to demonstrate as a control experiment.

Requirements:

- The chain must be configurable, not hardcoded into your logic.
- Store every intermediate result, not just the final one. The reveal animation and the hint system both depend on this.
- Handle the case where a hop fails or returns an empty string. Note that MyMemory reports some failures inside an otherwise successful HTTP response, so check the status field in the body and not only the HTTP code. Retry once, then abort the round cleanly and log it. Never let a half-finished chain reach a player.
- Seed sentences must be short enough to survive MyMemory's 500 byte limit. Cap them at 120 characters.
- Idioms make the best seeds because they are exactly what literal translation destroys. "It's raining cats and dogs", "break a leg", "the early bird catches the worm", "he let the cat out of the bag". Build a seed table of at least 40.

5. Frontend requirements

The reveal is the centrepiece. When a round closes, do not dump the seven steps as a static list. Animate the chain one hop at a time, showing the language flag or code, the text at that stage, and how far the text moved at that hop. For that last figure use the change in character count from the previous step, since a string similarity between two different scripts is not a meaningful number. Let it breathe: roughly a second per hop, so eight to ten seconds end to end. This is what the class will remember.

Also required:

- Live round timer.
- A guess input with instant feedback on submission (correct, close, or wrong) without a page reload.
- A hint button that reveals one intermediate step and visibly costs points, at the rate given in section 2.
- A Hall of Fame view, sortable by mangle score and by votes, where any player may upvote an entry once and that limit is enforced on the server.
- A loading state on round generation. Seven sequential API calls take real time, so tell the user something is happening.
- Correct behaviour on a phone-sized screen.